

OSSP PRACTICAL Q5

5. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Code:-

```
A) #include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    int fd[2];

    pid_t pid;

    char buffer[100];

    // Create pipe

    if (pipe(fd) == -1) {

        perror("Pipe");

        exit(1);

    }

    pid = fork();

    if (pid < 0) {

        perror("Fork");

        exit(1);

    }
```

```

if (pid == 0) {
    // Child - Consumer

    close(fd[1]); // Close write end

    read(fd[0], buffer, sizeof(buffer));

    printf("Child received: %s\n", buffer);

    close(fd[0]);
}

else {
    // Parent - Producer

    close(fd[0]); // Close read end

    char message[] = "Hello from Parent using Pipe!";

    write(fd[1], message, strlen(message) + 1);

    close(fd[1]);

    wait(NULL);
}

return 0;
}

```

```

B) #include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    int fd[2];

    if (pipe(fd) == -1) {

        perror("Pipe");
    }
}

```

```
    exit(1);
}

pid_t pid1 = fork();
if (pid1 == 0) {
    // First child: ls -l

    close(fd[0]);      // Close read end
    dup2(fd[1], STDOUT_FILENO);
    close(fd[1]);
    execlp("ls", "ls", "-l", NULL);
    perror("execlp");
    exit(1);
}

pid_t pid2 = fork();
if (pid2 == 0) {
    // Second child: grep ".c"

    close(fd[1]);      // Close write end
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    execlp("grep", "grep", ".c", NULL);
    perror("execlp");
    exit(1);
}

// Parent
close(fd[0]);
close(fd[1]);
wait(NULL);
```

```
wait(NULL);  
return 0;  
}
```

OUTPUT:-

```
practical — nano q5_pipe.c — 80x24
UW PICO 5.09 File: q5_pipe.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;
    char buffer[100];

    // Create pipe
    if (pipe(fd) == -1) {
        perror("Pipe");
        exit(1);
    }

    pid = fork();

    ^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
    ^X Exit      ^J Justify   ^W Where is   ^V Next Pg   ^U UnCut Text ^T To Spell

```

```
practical — nano q5_shellpipe.c — 80x24
UW PICO 5.09 File: q5_shellpipe.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("Pipe");
        exit(1);
    }

    pid_t pid1 = fork();

    if (pid1 == 0) {
        // First child: ls -l
        close(fd[0]); // Close read end

    }

    ^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
    ^X Exit      ^J Justify   ^W Where is   ^V Next Pg   ^U UnCut Text ^T To Spell

```

```
apple@apples-MacBook-Pro-2 practical % ./q5_pipe
Child received: Hello from Parent using Pipe!
```

```
apple@apples-MacBook-Pro-2 practical % ./q5_shellpipe
-rw-r--r-- 1 apple  staff   1079 Aug  5 12:34 practical.c
-rwxr-xr-x 1 apple  staff  33224 Aug  5 12:34 practicalq3
-rw-r--r-- 1 apple  staff   1056 Aug  7 12:35 practicalq3.c
-rwxr-xr-x 1 apple  staff  33400 Aug  5 12:36 practicalq4
-rw-r--r-- 1 apple  staff    975 Aug  5 12:36 practicalq4.c
-rwxr-xr-x 1 apple  staff  33360 Aug  5 12:37 practicalq4_waitpid
-rw-r--r-- 1 apple  staff    684 Aug  5 12:37 practicalq4_waitpid.c
-rwxr-xr-x 1 apple  staff  33312 Aug  7 12:35 practiq3
-rw-r--r-- 1 apple  staff    842 Aug  5 12:39 q5_pipe.c
-rw-r--r-- 1 apple  staff    879 Aug  5 12:40 q5_shellpipe.c
```